```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Dense,
    Flatten,
    Dropout,
    BatchNormalization,
    Conv2D,
    MaxPooling2D
)

from tensorflow.keras.applications import VGG16

import matplotlib.pyplot as plt
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
def create_vgg16_model(drop_rate):

    base_model = VGG16(
        weights='imagenet',
        include_top=False,
        input_shape=(48,48,3)
    )

    for layer in base_model.layers:
        layer.trainable = False

    model = Sequential()

    model.add(base_model)

    model.add(Flatten())

    model.add(Dense(128, activation='relu'))

    model.add(Dropout(drop_rate))

    model.add(Dense(10, activation='softmax'))

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
model_02 = create_vgg16_model(0.2)

history_02 = model_02.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=2,
    batch_size=64
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernel
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 30s 32ms/step - accuracy: 0.8589 - loss: 0.4871 - val_accuracy: 0.9476 - val_loss: 0.1843
Epoch 2/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.9464 - loss: 0.1791 - val_accuracy: 0.9615 - val_loss: 0.1274
```

```
model_08 = create_vgg16_model(0.8)

history_08 = model_08.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=2,
    batch_size=64
)
```

```
Epoch 1/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 29s 35ms/step - accuracy: 0.6699 - loss: 1.0056 - val_accuracy: 0.9225 - val_loss: 0.3339
Epoch 2/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 35ms/step - accuracy: 0.8225 - loss: 0.5550 - val_accuracy: 0.9426 - val_loss: 0.2167
```

```
def create_bn_model(drop_rate):

    base_model = VGG16(
        weights='imagenet',
        include_top=False,
        input_shape=(48,48,3)
    )

    for layer in base_model.layers:
        layer.trainable = False

    model = Sequential()

    model.add(base_model)

    model.add(Flatten())

    model.add(BatchNormalization())

    model.add(Dense(128, activation='relu'))

    model.add(Dropout(drop_rate))

    model.add(Dense(10, activation='softmax'))

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
bn_model_02 = create_bn_model(0.2)

bn_history_02 = bn_model_02.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=2,
    batch_size=64
)
```

```
Epoch 1/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 31s 37ms/step - accuracy: 0.9237 - loss: 0.2408 - val_accuracy: 0.9682 - val_loss: 0.0981
Epoch 2/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 35ms/step - accuracy: 0.9686 - loss: 0.0989 - val_accuracy: 0.9756 - val_loss: 0.0735
```

```
bn_model_05 = create_bn_model(0.5)

bn_history_05 = bn_model_05.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=2,
    batch_size=64
)
```

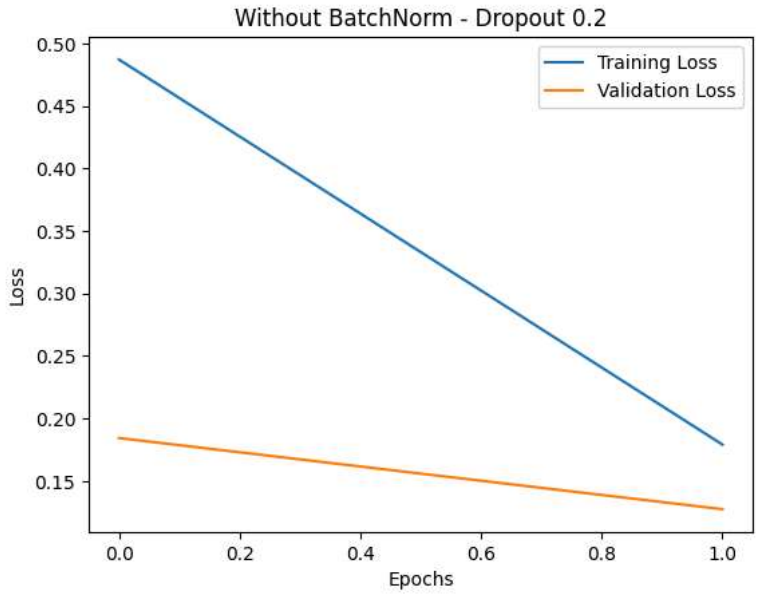
```
Epoch 1/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 30s 36ms/step - accuracy: 0.8987 - loss: 0.3260 - val_accuracy: 0.9678 - val_loss: 0.1067
```

```
Epoch 2/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 35ms/step - accuracy: 0.9539 - loss: 0.1476 - val_accuracy: 0.9728 - val_loss: 0.0832
```

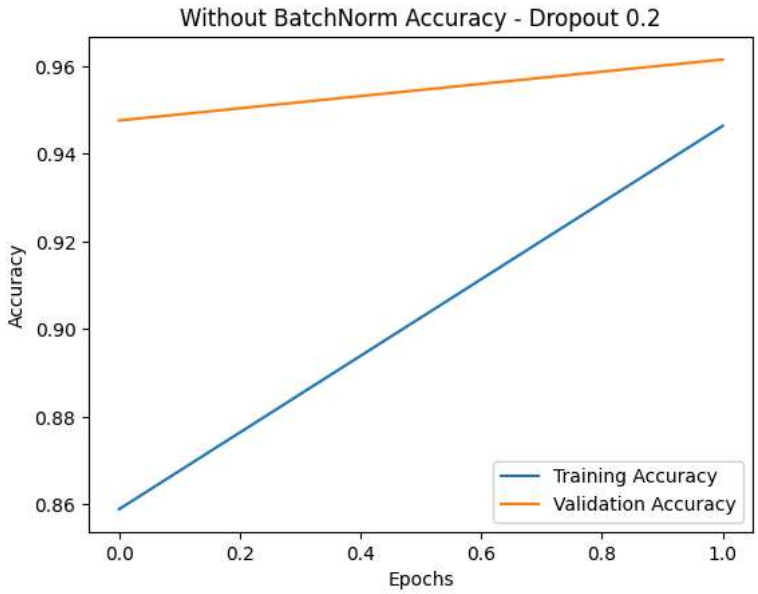
```
bn_model_08 = create_bn_model(0.8)

bn_history_08 = bn_model_08.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=2,
    batch_size=64
)
```

```
Epoch 1/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 30s 36ms/step - accuracy: 0.7967 - loss: 0.6473 - val_accuracy: 0.9552 - val_loss: 0.1477
Epoch 2/2
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 34ms/step - accuracy: 0.9010 - loss: 0.3190 - val_accuracy: 0.9647 - val_loss: 0.1157
```

```
plt.plot(history_02.history['loss'], label='Training Loss')
plt.plot(history_02.history['val_loss'], label='Validation Loss')

plt.title('Without BatchNorm - Dropout 0.2')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```




```
plt.plot(history_02.history['accuracy'], label='Training Accuracy')
plt.plot(history_02.history['val_accuracy'], label='Validation Accuracy')

plt.title('Without BatchNorm Accuracy - Dropout 0.2')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```




```
loss, accuracy = bn_model_05.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 21ms/step - accuracy: 0.9723 - loss: 0.0840
Test Loss: 0.0839616060256958
Test Accuracy: 0.9722999930381775
```

Start coding or generate with AI.